

halfedge：布尔运算模块设计文档

src/boolean.rs

halfedge 库

2026-07-04

目录

1 模块定位	1
1.1 API 总览	1
2 BoolOp 分类规则	2
3 核心算法：射线投射内外判定	2
3.1 Möller-Trumbore 射线-三角形求交	2
3.2 点在网格内外判定（射线奇偶法）	2
3.3 面分类：双侧法向偏移采样	3
3.4 整体流程	3
4 顶点去重	3
5 复杂度	3
6 限制	4
7 测试覆盖	4
8 设计取舍	4

1 模块定位

`boolean.rs` 在两个闭合流形三角网格 A 、 B 之上执行**集合运算**（并、交、差、对称差），返回新的 `MeshStorage`。

算法路线：基于射线投射的**逐面内外分类**（point-in-polyhedron by ray parity），而非 BSP / CSG 切割。每个输入三角形作为整体保留或丢弃，不进行跨越边界的几何切割。

1.1 API 总览

类别	API	语义
枚举	BoolOp	Union/Intersection/Difference/SymmetricDifference
核心	boolean_operation(a, b, op)	按 op 分派, 返回新 MeshStorage
快捷	boolean_union(a, b)	$A \cup B$
	boolean_intersection(a, b)	$A \cap B$
	boolean_difference(a, b)	$A - B$
	boolean_symmetric_difference(a, b)	$A \triangle B$

2 BoolOp 分类规则

classify(op, inside_a, inside_b) 根据 A 、 B 内外状态决定保留:

op	保留条件	集合表达
Union	inside_a inside_b	$A \cup B$
Intersection	inside_a && inside_b	$A \cap B$
Difference	inside_a && !inside_b	$A \setminus B$
SymmetricDifference	inside_a != inside_b	$A \triangle B$

3 核心算法：射线投射内外判定

3.1 Möller-Trumbore 射线-三角形求交

设射线 $\vec{R}(t) = \vec{O} + t\vec{D}$, 三角形 $(\vec{V}_0, \vec{V}_1, \vec{V}_2)$, 求解重心坐标参数 (t, u, v) :

$$\vec{O} + t\vec{D} = (1 - u - v)\vec{V}_0 + u\vec{V}_1 + v\vec{V}_2.$$

接受条件:

- $|\det| \geq 10^{-14}$ (非平行);
- $u \geq 0, v \geq 0, u + v \leq 1$ (在三角形内);
- $t > 10^{-12}$ (在前向半空间, 避免自交)。

3.2 点在网格内外判定 (射线奇偶法)

point_inside_point(point, mesh) 沿 $+x$ 方向发射射线, 统计与三角面交点数:

$$\text{inside}(\vec{p}) \iff \#\{\text{hits}\} \bmod 2 = 1.$$

假设网格闭合且法向一致。复杂度 $O(F)$ 。

3.3 面分类：双侧法向偏移采样

对每个面 f (中心 $\vec{c}$, 法向 $\vec{n}$), 为避免中心点恰好落在另一网格表面, 沿法向两侧各偏移 ε 采样:

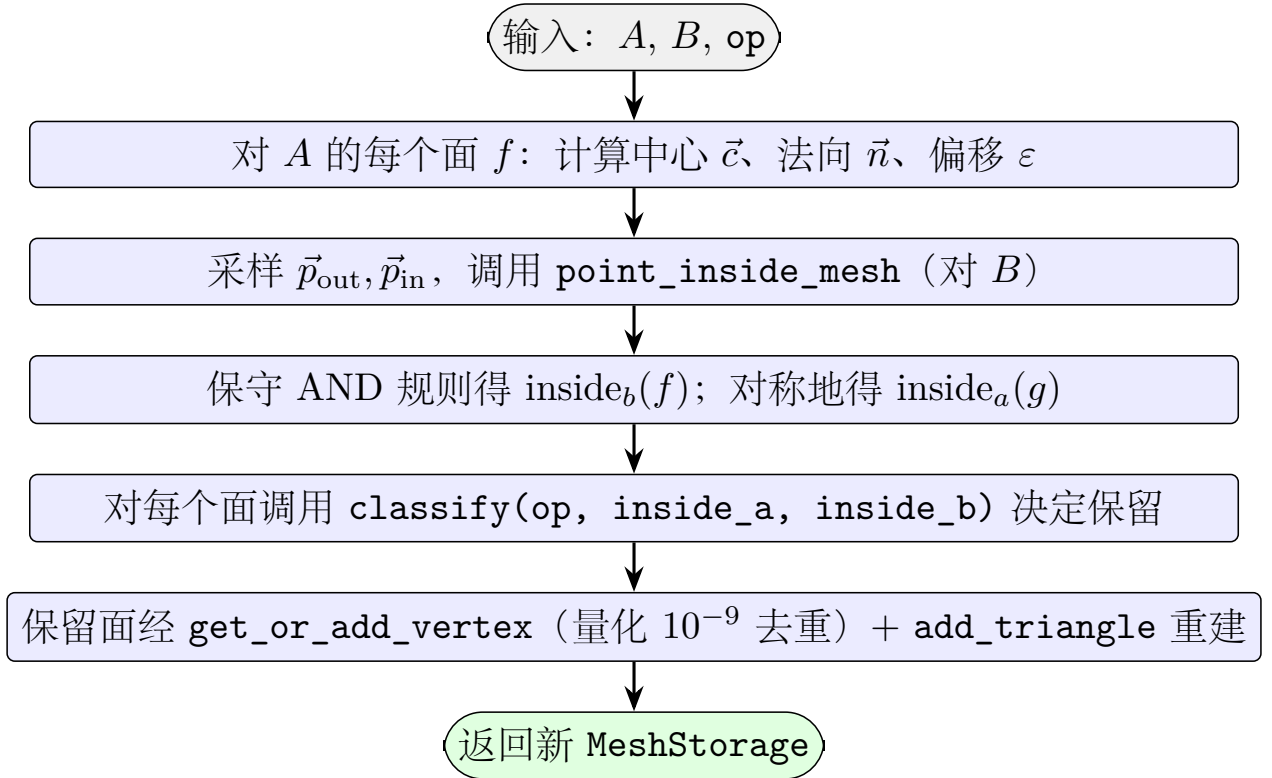
$$\vec{p}_{\text{out}} = \vec{c} + \varepsilon \hat{n}, \quad \vec{p}_{\text{in}} = \vec{c} - \varepsilon \hat{n},$$

其中 $\varepsilon = 10^{-4}/\|\vec{n}\|$ ($\|\vec{n}\| < 10^{-14}$ 时回退 $\varepsilon = 10^{-4}$)。

保守 AND 规则:

$$\text{inside} = \text{inside}(\vec{p}_{\text{in}}) \wedge \neg \text{inside}(\vec{p}_{\text{out}}).$$

3.4 整体流程



4 顶点去重

`get_or_add_vertex` 通过量化网格去重:

$$\text{key} = (\lfloor 10^9 \cdot x \rfloor, \lfloor 10^9 \cdot y \rfloor, \lfloor 10^9 \cdot z \rfloor) \in \mathbb{Z}^3.$$

`HashMap<[i64;3], VertexId>` 缓存已添加顶点; 阈值 10^{-9} 内的位置视为同一点。这保证保留三角形之间的共享顶点能被识别, 使 `add_triangle` 自动配对 twin。

5 复杂度

设 $F_a = \text{mesh_a.face_count}()$, $F_b = \text{mesh_b.face_count}()$, $K =$ 保留三角形数。

函数	时间复杂度
<code>point_inside_mesh</code>	$O(F)$ (遍历所有面求交)
<code>ray_triangle_intersection</code>	$O(1)$
<code>classify</code>	$O(1)$
<code>get_or_add_vertex</code>	$O(1)$ 均摊
<code>boolean_operation</code>	$O(F_a \cdot F_b)$ (每面两次 <code>point_inside_mesh</code>)
四个快捷函数	$O(F_a \cdot F_b)$ (委托 <code>boolean_operation</code>)

注意：复杂度在面数乘积上，对稠密网格不友好。无加速结构（BVH / 八叉树）。

6 限制

- 仅支持**闭合流形三角网格**；非闭合网格的内外判定无定义；
- 共面退化情形（两三角形共面）可能误判；
- 跨越边界的三角形**不被切割**：保留与否取决于其重心分类结果，对部分重叠网格是**近似的**（内部面正确移除，但边界接缝不重新三角化）；
- `add_triangle` 的 `Result` 被显式忽略 (`let _ = ...`)，拓扑错误被静默吞掉。

7 测试覆盖

测试名	验证点
<code>union_disjoint_cubes</code>	不相交两立方体并集： $F = 24$ （无丢弃）
<code>union_overlapping_cubes</code>	重叠两立方体并集： $4 \leq F \leq 24$ （内部面被丢弃）
<code>intersection_overlapping_cubes</code>	重叠立方体交集非空 ($V > 0$)
<code>intersection_disjoint_cubes</code>	不相交立方体交集为空 ($F = 0$)
<code>difference_self_is_empty</code>	$A - A$ 近似空 ($F \leq 2$ ，容差考虑射线伪影)
<code>symmetric_difference_disjoint_equals_union</code>	不相交时对称差面数 = 并集面数
<code>intersection_contains_cube</code>	两相同立方体交集 ≥ 6 面

全模块共 **7** 个单元测试，全库共 **371** 个。

8 设计取舍

- **为何用射线投射而非 BSP / CSG 切割？** 实现简洁，无需计算交线、重新三角化跨越边界的面。对完全包含 / 完全不相交的情形结果精确；部分重叠是近似。适合快速原型与几何查询。

- **为何两侧法向偏移采样？** 重心可能恰好落在另一网格表面（如两立方体共面时），单侧采样会产生歧义。两侧采样 + AND 规则提供保守的鲁棒判定。
- **为何 `add_triangle` 的错误被忽略？** 布尔结果中可能出现退化三角形（共线顶点）或重复面，`add_triangle` 会返回 `Err`。当前实现选择丢弃错误三角形继续，保证调用方拿到非空结果。生产环境应改为收集错误并报告。
- **为何顶点量化去重？** 浮点位置直接比较不可靠（ 10^{-16} 误差）。量化到 10^{-9} 网格既保证共享顶点被识别，又允许微小数值漂移。